Análise Técnica — MoovePay

Data: 27 de abril de 2026
Escopo: Revisão completa de arquitetura, banco de dados, multi-tenant e segurança

---

1. Visão Geral

Stack:

- Next.js 14 (App Router) + TypeScript + Tailwind CSS + shadcn/ui
- Supabase (PostgreSQL via PostgREST) + `SERVICE_ROLE_KEY` server-only
- Autenticação customizada: JWT HS256 em cookie `httpOnly`
- Resend (e-mail transacional)
- ASAAS (gateway de pagamentos — integração incompleta)
- Zustand (estado do carrinho no cliente)

Propósito: Plataforma SaaS multi-tenant para criadores de eventos gerenciarem inscrições, pagamentos, vouchers e check-in de participantes.

---

2. Banco de Dados

2.1 Tabelas e Status

Tabela	Propósito	Status
`User`	Usuários da plataforma	✅ Sólido
`Tenant`	Conta / organização	⚠️ Migration criada, aplicação pendente
`TenantMember`	Membros de cada tenant com roles	⚠️ Migration criada, aplicação pendente
`Event`	Eventos dos criadores	✅ Com coluna `tenantId` via migration
`InscriptionType`	Tipos de inscrição por evento	✅ Sólido
`Registration`	Inscrições dos participantes	✅ Sólido
`Payment`	Pagamentos	✅ Estrutura pronta, integração incompleta
`Voucher`	QR code de voucher	✅ Estrutura pronta, geração não implementada
`SupportTicket`	Tickets de suporte	✅ Bem implementado (SLA, tags, assignment)
`SupportTicketMessage`	Mensagens de ticket	✅ Com trigger automático de SLA

2.2 Migrations (ordem cronológica)

Arquivo	Conteúdo	Avaliação
`20260202000000_init.sql`	Schema base (User, Event, Registration, Payment, Voucher, TicketConfig)	✅ OK
`20260203000000_support_tickets.sql`	SupportTicket + SupportTicketMessage	✅ OK
`20260203001000_multitenant_rbac.sql`	Tenant, TenantMember, tenantId nas tabelas + backfill	⚠️ Ver nota abaixo
`20260203002000_support_ticket_assignment.sql`	Campo `assignedToUserId` no ticket	✅ OK
`20260203003000_support_ticket_sla_tags.sql`	Campos SLA, tags, trigger `support_ticket_on_message_insert`	✅ OK
`20260204000000_user_profile.sql`	Colunas `avatarUrl` e `address` no User	✅ OK
`20260204001000_storage_bucket_media.sql`	Documental — bucket criado automaticamente pelo app	ℹ️ OK

⚠️ **Nota — migration `20260203001000`:**
O trecho `ALTER TABLE "Event" ALTER COLUMN "tenantId" SET NOT NULL` pode falhar em produção se houver eventos cujo `creatorId` não corresponde a nenhum `User` existente (FK quebrada no backfill). Recomenda-se executar o backfill em transação e validar antes de aplicar o NOT NULL.

2.3 Índices Ausentes

Os índices abaixo não foram criados nas migrations mas são necessários para performance:

Tabela	Coluna	Justificativa
`Registration`	`cartId`	Lookup de checkout em lote
`Payment`	`cartId`	Join com registrations por carrinho

---

3. Multi-Tenant

3.1 Arquitetura Adotada

```
User ──┬── TenantMember (role: owner/member) ─── Tenant
       └── defaultTenantId                          │
                                                    ├── Event (tenantId)
                                                    └── SupportTicket (tenantId)
```

O modelo adota **1 usuário = 1 tenant padrão** no cadastro (backfill: `tenantId = userId`), com estrutura preparada para múltiplos membros por tenant no futuro.

3.2 O Que Está Implementado

- ✅ Tabelas `Tenant` e `TenantMember` criadas

- ✅ `User.defaultTenantId` adicionado
- ✅ `Event.tenantId` e `SupportTicket.tenantId` com FK para `Tenant`
- ✅ `lib/rbac.ts` com `getAuthContext()` e `isTenantMember()`
- ✅ `verifyToken()` inclui `tenantId` no payload JWT
- ✅ Login injeta `tenantId = defaultTenantId || userId` no token

3.3 Problemas Identificados

Fallback legado espalhado por todo o código
A função `isTenantMember()` retorna `{ id: 'legacy' }` quando a tabela `TenantMember` não existe no banco. As APIs `/api/events`, `/api/events/[id]` e `/api/support/tickets` verificam `member?.id === 'legacy'` e trocam o filtro de `tenantId` para `creatorId`. Isso indica que as migrations **não foram aplicadas em produção**.

Verificação de tenant tardia em `GET /api/events/[id]`
O evento é buscado sem filtro de `tenantId` na query; a verificação ocorre apenas depois de receber o resultado:

```
// Busca sem filtro:
.eq('id', params.id)
// Verifica DEPOIS:
if ((event as any).tenantId && (event as any).tenantId !== tenantId) { ... }
```

Se `event.tenantId` for `null` (dado legado), a verificação é pulada e qualquer usuário autenticado pode ler qualquer evento.

---

4. Autenticação e Segurança

4.1 O Que Está Correto

- ✅ JWT HS256 com cookie `httpOnly + secure + sameSite: strict`
- ✅ Validade de 7 dias com verificação de expiração
- ✅ Verificação JWT implementada manualmente com Web Crypto API no Edge Middleware (compatível com Edge Runtime, sem `jsonwebtoken`)
- ✅ Senhas com bcrypt (salt 10)
- ✅ Validação Zod em todos os endpoints
- ✅ `lib/supabase-server.ts` marcado com `'server-only'` — não vaza para o cliente
- ✅ Supabase usa `SERVICE_ROLE_KEY` (bypass RLS controlado pelo servidor)
- ✅ Google OAuth com parâmetro `state` anti-CSRF em cookie `httpOnly`

4.2 Problemas de Segurança

🔴 **CRÍTICO — JWT_SECRET com fallback inseguro**

```
// lib/auth.ts
const JWT_SECRET = process.env.JWT_SECRET || 'seu-secret-jwt'
```

Em qualquer ambiente onde `JWT_SECRET` não estiver definido, todos os tokens são assináveis com o valor padrão `'seu-secret-jwt'`. Um atacante pode forjar tokens válidos.

Correção: Lançar erro se a variável não estiver presente:

```
const JWT_SECRET = process.env.JWT_SECRET
if (!JWT_SECRET) throw new Error('JWT_SECRET não definido')
```

🔴 CRÍTICO — `GET /api/registrations` sem autenticação

O endpoint lista inscrições por `email` (query param) sem verificar token JWT. Qualquer pessoa pode consultar dados pessoais (nome, CPF, WhatsApp) de qualquer participante conhecendo apenas o e-mail.

Correção: Adicionar verificação de token no início do `GET handler`, ou restringir a consulta ao e-mail do próprio usuário autenticado.

🔴 CRÍTICO — ADMIN_ACCESS_CODE não verificado

O PLANEJAMENTO descreve um `ADMIN_ACCESS_CODE` como requisito para login de staff. A rota `/api/admin/auth/login` verifica apenas se o `role` está em `['admin', 'support', 'finance']`, sem validar o código extra. Qualquer credencial com role staff no banco é suficiente para acessar o painel admin.

⚠️ IMPORTANTE — `NEXT_PUBLIC_ASAAS_API_URL` exposta ao cliente

A variável `NEXT_PUBLIC_ASAAS_API_URL` é enviada ao bundle do navegador. Embora a chave `ASAAS_API_KEY` seja server-only, a URL base da API não precisa ser pública.

Correção: Renomear para `ASAAS_API_URL` e usar apenas server-side.

⚠️ IMPORTANTE — Role lida do `localStorage` no frontend

O `DashboardLayout` e `AdminHomePage` leem `role` do `localStorage` para controlar quais itens de menu exibir:

```
const raw = localStorage.getItem('user')
const parsed = JSON.parse(raw)
setRole(String(parsed?.role || 'user'))
```

Um usuário pode manipular o `localStorage` para visualizar menus de admin — embora as APIs validem server-side, isso gera confusão e pode facilitar ataques de engenharia social.

Correção: Buscar dados do usuário via `GET /api/auth/me` e não confiar no `localStorage` para decisões de autorização no frontend.

---

5. APIs — Mapa de Qualidade

Endpoint	Auth	Tenant Check	Validação Zod	Observação
`POST /api/auth/login`	N/A	N/A	✅	OK
`POST /api/auth/signup`	N/A	N/A	✅	OK
`GET /api/auth/google`	N/A	N/A	N/A	OK (state anti-CSRF)
`POST /api/admin/auth/login`	N/A	N/A	✅	⚠️ Falta ADMIN_ACCESS_CODE
`GET /api/events`	✅ JWT	✅ (fallback)	—	Fallback legado pendente
`POST /api/events`	✅ JWT	✅ (fallback)	✅	OK
`GET /api/events/[id]`	✅ JWT	⚠️ Tardio	—	Tenant check pós-query
`PUT /api/events/[id]`	✅ JWT	✅	✅	OK
`DELETE /api/events/[id]`	✅ JWT	✅	—	OK
`POST /api/registrations`	❌ Público	❌ Nenhum	✅	Correto ser público (inscrição pública)
`GET /api/registrations`	❌ **Sem auth**	❌ Nenhum	—	🔴 **Expõe dados sem autenticação**
`GET/POST /api/support/tickets`	✅ JWT	✅ (fallback)	✅	Fallback legado
`GET /api/admin/tenants`	✅ JWT	✅ role=admin	—	OK

---

6. Módulo de Pagamentos (ASAAS)

Status atual: Estrutura pronta, integração não implementada.

Componente	Status
Cliente HTTP ASAAS (`lib/asaas.ts`)	✅ Criado (PIX, cartão, boleto)
Tabela `Payment` no banco	✅ Criada
Endpoint de checkout que cria cobrança	❌ Não existe
Webhook para receber confirmação de pagamento	❌ Não existe

Componente	Status
Atualização de `Registration.status` para `paid`	❌ Não implementada
Geração de voucher/QR após pagamento confirmado	❌ Não implementada
Envio de voucher por e-mail	❌ Templates criados, envio não acionado
Check-in por QR (validação)	❌ Não implementado

O fluxo atual termina com `Registration.status = 'pending'` permanentemente — inscrições nunca avançam para `paid`.

---

7. Funcionalidades Ausentes (Escopo do MVP)

Funcionalidade	Prioridade	Status
Pagamento ASAAS (PIX / cartão / boleto)	Alta	❌
Webhook de confirmação de pagamento	Alta	❌
Geração de voucher + QR code	Alta	❌
Envio de voucher por e-mail	Alta	❌
Check-in por QR	Média	❌
Convite de membro ao tenant	Baixa	❌
Reenvio de voucher (dashboard/admin)	Baixa	❌

---

8. Dependências Não Utilizadas

Pacote	Situação
`next-auth`	Instalado mas **não utilizado** — auth é customizada com JWT
`pg`	Instalado, não há uso direto de `pg` (tudo via Supabase client)

Recomendação: Remover esses pacotes para reduzir a superfície de ataque e o tamanho do bundle.

---

9. Prioridades de Correção

🔴 Crítico — Segurança (ação imediata)

1. `JWT_SECRET` **sem fallback** — remover o valor padrão `'seu-secret-jwt'` e exigir a variável de ambiente
2. **Autenticar** `GET /api/registrations` — o endpoint expõe dados pessoais (nome, CPF, WhatsApp) sem nenhum token
3. **Tenant check na query** em `GET /api/events/[id]` — mover a verificação para `.eq('tenantId', tenantId)` na query do banco
4. **Implementar ou remover** `ADMIN_ACCESS_CODE` — definir se é requisito de negócio e agir

🟡 Importante — Funcionalidade

5. **Aplicar migrations no banco de produção** e remover todos os fallbacks legados do código
6. **Implementar fluxo de pagamento completo**: ASAAS → webhook → update `Registration.status` → gerar `Voucher` → enviar e-mail
7. **Adicionar índices** em `Registration.cartId` e `Payment.cartId`

🟢 Melhoria — Qualidade e Manutenção

8. **Substituir leitura de `role` do `localStorage`** por dados vindos de `GET /api/auth/me`
9. **Remover `next-auth` e `pg`** do `package.json`
10. **Renomear `NEXT_PUBLIC_ASAAS_API_URL`** para `ASAAS_API_URL` (server-only)

---

10. Resumo Executivo

O MoovePay tem uma base técnica bem estruturada: schema de banco pensado para crescimento, autenticação JWT com boas práticas (cookie `httpOnly`, bcrypt, validações Zod), e arquitetura multi-tenant funcional com fallback seguro para ambientes ainda não migrados.

Os três pontos que bloqueiam o produto:

1. **Segurança:** `JWT_SECRET` sem proteção e endpoint de registrations público expondo dados pessoais são riscos imediatos.

2. **Multi-tenant incompleto:** As migrations precisam ser aplicadas em produção. Enquanto o fallback legado estiver ativo, o isolamento entre tenants não está sendo enforçado.

3. **Pagamentos:** Toda a cadeia ASAAS → confirmação → voucher → e-mail está ausente, o que impede o produto de funcionar como plataforma de cobranças.

Análise realizada em 27/04/2026 — GitHub Copilot